



MÓDULO 4 · AULA 30

Aula 30

Automação de testes na análise de dados

A pirâmide aplicada a DS: unitário, API do modelo e interface.

Instrutor: Douglas Adriano Queiroz

O que vamos ver hoje



Pirâmide em DS

Onde cada teste entra



Unitário

Funções e validação de dado



API do modelo

Servir e testar previsões



Interface

Dashboards e relatórios



CI

Rodar tudo na esteira



Boas práticas

Automação que se sustenta

A pirâmide de testes aplicada a Data Science

A mesma pirâmide da Aula 9, adaptada ao mundo dos dados e modelos.

Base — unitário + validação de dado: funções de limpeza (pytest) e expectativas do dataset (pandera/GE).

Meio — API do modelo: o modelo servido por uma API responde certo? (testes de integração).

Topo — interface/E2E: o dashboard/relatório mostra os números certos ao usuário.

Muitos embaixo, poucos no topo: rápido, barato e confiável.



Analogia da cozinha

Provar cada ingrediente (unitário), provar o molho já misturado (API/integração) e, por fim, provar o prato montado no prato do cliente (interface). Cada nível pega um tipo de erro.



Leve daqui: Em DS a pirâmide inclui um nível extra: a validação do próprio dado.

Os níveis, na prática



Unitário (pytest)

Funções de limpeza/feature: entrada conhecida → saída esperada.



Validação de dado

Schema, faixas, nulos e duplicados (pandera / Great Expectations).



API do modelo

POST com dados → retorna previsão no formato certo e a tempo.



Interface / dashboard

Filtros, totais e gráficos mostram os números corretos.

Testar a API do modelo

Modelos costumam ser servidos por uma API (FastAPI/Flask): o cliente envia dados e recebe a previsão.

Endpoint: ex.: POST /prever recebe as features e devolve a classe/probabilidade.

O que validar: status 200, formato da resposta, faixa (probabilidade entre 0 e 1).

Negativos: dados faltando ou inválidos devem retornar 400, não quebrar.

Ferramentas: pytest + requests, ou Postman (lembra a Aula 14).



Analogia da casa

O modelo é a 'casa' e o endpoint /prever é a porta. Testamos: a porta abre e entrega a previsão certa? tranca para pedido inválido? responde rápido? É teste de API, igual ao da Aula 14.



Leve daqui: Servir o modelo é uma API — teste como qualquer API: status, formato, borda e tempo.

Exemplo: testar o endpoint de previsão

```
test_api_moodleb.py

import requests

def test_prever_retorna_probabilidade_valida():
    cliente = { 'tem_po_casa': 12, 'valor_mensal': 89.9, 'chamados': 3 }
    r = requests.post('http://localhost:8000/prever', json=cliente)
    assert r.status_code == 200
    corpo = r.json()
    assert 0.0 <= corpo['probabilidade'] <= 1.0 # faixa válida

def test_prever_dado_invalido_retorna_400():
    r = requests.post('http://localhost:8000/prever', json={})
    assert r.status_code == 400
```



Leve daqui: A previsão é testada por faixa (0-1) e pelo comportamento com dado inválido.

Interface, dashboards e CI

Fechamos testando o que o usuário vê e rodando tudo automaticamente.

Dashboard/BI: conferir se filtros e totais batem com o dado (Power BI, Streamlit).

E2E: um teste que abre o dashboard e valida um número-chave (Playwright).

CI: GitHub Actions roda pytest + validação de dados a cada push.

Notebooks no CI: ferramentas como nbmake executam notebooks para garantir que rodam.



Analogia da casa

A esteira de qualidade da fábrica: nada sai para o cliente sem passar por todas as estações de teste, automaticamente, a cada lote.



Leve daqui: Automatize do dado ao dashboard, na esteira — feedback rápido a cada mudança.



Resumão da aula



A pirâmide em DS: unitário + validação de dado (base), API (meio), interface (topo).



Unitário (pytest) prova funções; pandera/GE validam o dataset.



O modelo é servido por API — teste status, formato, borda e tempo.



Interface/dashboard: filtros e totais precisam bater com o dado.



CI (GitHub Actions) roda testes e validação a cada push; nbmake roda notebooks.



Muitos testes rápidos embaixo, poucos e caros no topo.



Glossário da aula

Pipeline de dados — Sequência: coletar → limpar → validar → modelar → servir

FastAPI / Flask — Frameworks Python para criar APIs (servir o modelo)

Endpoint — Porta da API (ex.: POST /prever)

Integração — Teste de peças funcionando juntas (API + modelo)

Dashboard — Painel visual com métricas e gráficos

Streamlit — Ferramenta Python para criar apps/dashboards de dados

CI — Integração contínua — roda testes a cada mudança

nbmake — Executa notebooks Jupyter dentro dos testes/CI

Feature — Variável de entrada usada pelo modelo

Latência — Tempo de resposta (ex.: da previsão)



Referências e para aprofundar

FastAPI / GitHub Actions. Servir modelos e automatizar testes no CI.

Docs oficiais. pandas.pydata.org · scikit-learn.org · docs.pytest.org · greatexpectations.io · rapidminer.com

Trilha do curso: Data Science e Testes — Projeto Social Garapuvu 2026.